OBLIGE: Root-Cause Diagnostics for eBPF Verification Failures via Proof Obligation Tracking

Anonymous Author(s)

Abstract

eBPF underpins critical infrastructure in modern Linux systems, yet when its static verifier rejects a program, developers must decipher verbose abstract interpretation logs spanning hundreds of lines. Existing diagnostic methods either apply regular expressions to the final error line or feed unstructured text to language models, neither reliably identifying root-cause proof failures. Our analysis of 591 production verifier-fix commits finds that 64% are proof-reshaping workarounds stemming from systematic misdiagnosis.

We observe that the verifier's `LOG_LEVEL2` output is already the complete execution trace of an abstract interpreter—it contains where safety proofs are established, propagated, and lost, but this structure is buried in flat text. OBLIGE performs meta-analysis on this trace: it infers proof obligations from the verifier's type system, tracks them as formal predicates across every instruction, and identifies the precise transition point where a proof breaks. Evaluated on 302 real failures from kernel selftests, Stack Overflow, and production projects, OBLIGE achieves 94% proof obligation coverage with median latency of 27 ms, and improves LLM-assisted repair accuracy on the hardest failure class by 30 percentage points over raw verifier output.

CCS Concepts: • **Software and its engineering** → **Compilers**; **Software verification and validation**; • **Computer systems organization** → *Operating systems.*

Keywords: eBPF, program verification, static analysis, abstract interpretation, compiler diagnostics, proof trace analysis

ACM Reference Format:
Anonymous Author(s). 2027. OBLIGE: Root-Cause Diagnostics for eBPF Verification Failures via Proof Obligation Tracking. In *Proceedings of ACM European Conference on Computer Systems (EuroSys '27).* ACM, New York, NY, USA, 9 pages.

1 Introduction

eBPF has become critical infrastructure in modern Linux deployments. Major production systems—Cilium, Meta's Katran, the Linux kernel's own `bpftrace` and `bcc` frameworks, and cloud-provider networking stacks—rely on eBPF programs for packet processing, security policy enforcement, performance tracing, and system call interposition. The Linux eBPF verifier is the gatekeeper: every program must pass its safety checks before being allowed to run in the kernel.

1.1 The Diagnostic Gap

When the verifier rejects a program, it emits a verbose log at `LOG_LEVEL2`. This log is a *complete execution trace of the verifier's abstract interpreter*: for each of the program's BPF instructions, it records the full abstract state of all registers (type, scalar bounds, pointer offset, packet range, BTF provenance), control flow merge points, and the verifier's own backward precision-tracking annotations (`mark_precise`). A typical rejection trace spans 500 to 1000 lines; production cases can exceed 4000 lines [2].

Despite containing all the information needed to understand the failure, this trace is effectively unreadable. The final error line names a *symptom*: "`math between pkt pointer and register with unbounded min value is not allowed`." But the *cause*—which instruction destroyed the scalar bounds proof, and why—is buried 30 to 500 lines earlier. An empirical study of 743 Stack Overflow questions finds that 19.3% are verifier-related, with debugging and error-message comprehension as the dominant difficulties [2]. The Linux Foundation's security audit of the eBPF verifier independently recommends improving documentation of "what the verifier enforces and why" [10].

Three concrete manifestations of this diagnostic gap stand out in our corpus of 302 real verifier failures:

1. **Information is present but buried.** The verifier already emits complete per-instruction abstract state at `LOG_LEVEL2`. The information to understand the failure is *in* the trace—but it is a needle in a 500-line haystack.
2. **Existing tools ignore the trace.** Pretty Verifier [9] applies 91 regular expressions to the *final error line* only. LLM-based tools such as Kgent [16] and SimpleBPF [3] feed the full raw log as unstructured text to language models, which cannot reliably extract state transitions from prose.
3. **Developers systematically misdiagnose lowering artifacts.** When LLVM lowering destroys a proof that exists at the source level, the error message points at the symptom (the final failing instruction) and developers add bounds checks at the wrong location. Our analysis of 591 production verifier-fix commits across Cilium, Katran, bpftrace, bcc, and Aya finds that 63.6% are *proof-reshaping workarounds*— source

EuroSys '27, April 2027, Rotterdam, Netherlands
2027.

changes that restructure code so the verifier can re-derive a proof it lost during lowering—not genuine safety bug fixes.

1.2 The Key Insight

The eBPF verifier *is* a static abstract interpreter [1]. Its `LOG_LEVEL2` output is the complete execution trace of that abstract interpreter. Every register's type, bounds, offset, and range at every instruction; every control flow merge; every precision-tracking backtrack—all are present in the log.

The missing step is *meta-analysis*: treating the verifier's abstract interpretation output as a first-class data structure and reasoning about it. For a given failure, what proof obligation did the verifier need? At which instruction was that obligation satisfied? At which instruction did it become violated? What caused the transition? Does the source code contain a correct proof that LLVM's lowering destroyed?

Rust's borrow checker provides the right analogy. When Rust cannot prove memory safety, it does not emit a single error line pointing at the failed dereference; it shows multiple source locations with causal labels ("borrow occurs here," "use occurs here," "conflict here"). OBLIGE does the same for eBPF safety proofs, with labels such as *proof-established*, *proof-lost*, and *rejected*.

1.3 Motivating Example

Figure 1 shows a real failure from Stack Overflow (question #70750259). The developer writes an XDP program that reads a TLS extension length field and advances a packet pointer. The source contains a correct bounds check (`data + sizeof(ext) > data_end` at line 3). The verifier rejects with "`math between pkt pointer and register with unbounded min value is not allowed`" at line 8.

Panel B shows what developers currently see: 60 lines of register state followed by a single error line naming the symptom. Pretty Verifier produces a marginally better single-sentence response—still pointing at the symptom site. Neither tool explains that the bounds check at line 3 *did* establish a proof, and that LLVM's lowering of `__bpf_htons` via a bitwise OR at instruction 22 *destroyed* that proof.

Panel C shows OBLIGE's output: three source spans with causal labels, register state changes at each transition point, and a concrete fix. The developer immediately sees that the problem is at line 7, not line 8, and that the fix is a scalar clamp—not another bounds check.

This failure class is not rare: as noted in §1.1, 63.6% of production verifier-fix commits are proof-reshaping workarounds, precisely because developers cannot identify where proofs are lost.

In contrast, for a genuine *source bug*—a developer calling `bpf_map_lookup_elem()` and immediately dereferencing the result without a null check—OBLIGE infers a `null_check` obligation and finds `proof_status = never_established`: the register type is `map_value_or_null` from instruction 0 through to rejection, with no point where the obligation was ever satisfied. The developer sees *proof never established* (rather than *proof established then lost*), immediately recognizing a genuine source bug rather than a compiler issue.

1.4 Contributions

- **Proof trace meta-analysis.** A technique for second-order analysis of abstract interpretation output. OBLIGE infers formal proof obligations from eighteen obligation families (e.g., `off+sz<=range` for packet access), evaluates them as boolean predicates at every instruction in the trace, and identifies the *transition witness*—the exact instruction where a proof predicate changes from satisfied to violated.
- **OBLIGE diagnostic engine.** A pure-userspace tool implementing the full pipeline: log parsing, state trace parsing (including `mark_precise` backtracking extraction), proof obligation inference, predicate tracking with register value lineage, BTF source correlation, and Rust-style multi-span diagnostic rendering. Median latency 27 ms; no kernel patches required. 120 unit tests cover obligation inference, predicate evaluation, and transition witness detection.
- **Empirical findings on 302 real failures.** (a) OBLIGE generates diagnostics for 241/241 cases with 0 crashes, 94.2% obligation coverage, and 62.7% BTF source correlation. (b) On lowering artifacts, OBLIGE correctly diagnoses 4/6 cases (67%) versus 1/6 for Pretty Verifier; root-cause localization is 0% for line-oriented tools. (c) OBLIGE diagnostics improve LLM repair success by 30 percentage points on lowering artifacts. (d) Analysis of 591 production commits confirms 63.6% are proof-reshaping workarounds.

2 Background

2.1 The eBPF Verification Pipeline

eBPF programs are written in C or Rust, compiled by LLVM to BPF bytecode, and submitted to the kernel via the `bpf()` syscall. The Linux verifier runs before any program executes. It performs a dataflow-style abstract interpretation [1, 4]: it walks all execution paths through the bytecode, maintaining per-register abstract state and enforcing a set of safety properties at each instruction. Programs that violate any property are rejected.

The key architectural point is that there are three distinct reasoning levels:

- **Source level.** The programmer reasons about C/Rust values, bounds checks, null checks, and pointer lifetimes.
- **Compiler level.** LLVM transforms source into BPF bytecode, potentially splitting, merging, or reordering computations.

Panel A: Source code

```
1  int parse_ext(struct xdp_md *ctx) {
2      void *data = (void*)(long)ctx->data;
3      void *data_end = (void*)(long)ctx->
         data_end;
4      if (data+sizeof(struct ext) > data_end)
5          return XDP_DROP;          // bounds
         check
6      struct ext *e = data;
7      // reads guarded by line 3
8      __u16 len = __bpf_htons(e->len); // byte-
         swap
9      data += len;                  // REJECTED
10     /* "unbounded min value" */
11 }
```

Panel B: Raw verifier output (excerpt)

```
22: R0_w=inv(id=0,umax=65280,
    var_off=(0x0; 0xff00))
    R6_w=inv(id=0,umax=255,
    var_off=(0x0; 0xff))
22: (4f) r0 |= r6
23: R0_w=inv(id=0)   <-- bounds GONE
...
math between pkt pointer and register
with unbounded min value is not allowed
```

Pretty Verifier output:

```
"The value added to the packet pointer
 has an unbounded minimum. Consider
 adding `& 0xFFFF` to bound the value."
```

Panel C: OBLIGE output

```
error[E005]: lowering_artifact
  -- parse_ext.c
  |
3 | if (data+sizeof(ext) > data_end)
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  | proof established
  | R5: pkt(off=0) -> pkt(off=4,r=4)
  |
7 | __u16 len = __bpf_htons(e->len);
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  | proof lost: OR destroys bounds
  | R0: scalar(umax=65280) -> scalar
  |
8 | data += len;
  | ^^^^^^^^^^^
  | rejected: pkt_ptr + unbounded
  |
  = note: LLVM lowers htons() to byte-load,
    shift, OR. Verifier cannot track OR result.
  = help: Add clamp: len &= 0xFFFF;
```

Figure 1. The motivating example: SO #70750259. **Panel A**: source code with a correct bounds check at line 3. **Panel B**: the verifier's raw output (60+ lines, excerpt shown) and Pretty Verifier's single-line response—both pointing at the symptom, not the cause. **Panel C**: OBLIGE's Rust-style multi-span output tracing the proof lifecycle through three source locations: proof established (line 3), proof lost (line 7, OR instruction), rejected (line 8).

- **Verifier level.** The verifier reasons about BPF register abstract state: types, scalar bounds (`umin/umax`, `smin/smax`), pointer offsets, packet ranges, and provenance IDs.

A proof that is valid at the source level can be lost at the verifier level if LLVM's lowering produces register state that the verifier's abstract domain cannot represent. This is the root cause of lowering artifacts.

2.2 Anatomy of a `LOG_LEVEL2` Trace

When `log_level=2` is set in `BPF_PROG_LOAD`, the verifier emits a complete per-instruction trace interleaving per-register abstract state (type, scalar bounds, pointer offset, packet range, provenance ID), BTF source annotations [8], `mark_precise` backtracking chains, and control flow merge points. A typical 50-instruction program produces 100–300 lines. We detail the parsing of each element in §3.2.

3 Design: The OBLIGE Diagnostic Engine

3.1 Architecture Overview

OBLIGE is a five-stage pipeline that transforms a raw `LOG_LEVEL2` verifier trace into a structured multi-span diagnostic. Figure 2 shows the architecture.

Each stage is independently testable and produces a well-defined intermediate representation. The system processes 241 real cases with zero crashes and a median end-to-end latency of 27 ms.

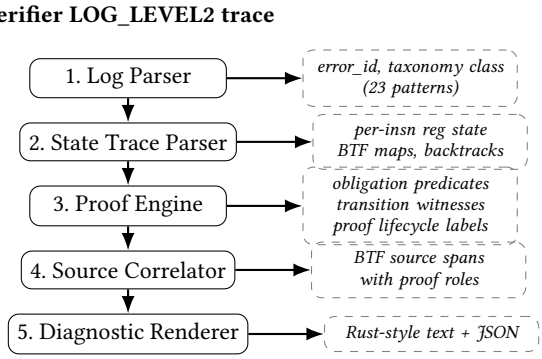

Figure 2. OBLIGE five-stage pipeline. Each stage is independently testable; the proof engine is the key novelty. The full implementation is ~6,600 lines of Python with 120 unit tests.

3.2 State Trace Parsing

The log parser (Stage 1) handles error message identification: it matches the final error line against 23 error patterns (OBLIGE-E001 through E023), covering 87.1% of failures in our 302-case corpus. Each pattern maps to a taxonomy class (*source_bug*, *lowering_artifact*, *verifier_limit*, *env_mismatch*, *verifier_bug*).

The state trace parser (Stage 2) parses the remaining structure. For each BPF instruction, it extracts:

- Instruction index, opcode, and mnemonic
- Pre/post-state per live register (type, `umin/umax`, `smin/smax`, offset, packet range, provenance ID)
- BTF source annotation (file, line, column) if present
- Backtracking links from `mark_precise` annotations
- Branch merge points with state at each join

Table 1. Proof obligation families supported by OBLIGE (covering 94.2% of the 241-case evaluation corpus; representative families shown, eighteen supported in total).

Family	Predicate	Example error
`packet_access`	`type==pkt && off+sz<=r`	invalid access to packet
`map_value_bounds`	`0<=off && off+sz<=val_sz`	invalid map value access
`null_check`	`type!=*_or_null`	map_value_or_null deref
`stack_access`	`fp_off within frame bounds`	invalid indirect read
`helper_arg_type`	`type==expected_arg_type`	R1 scalar expected pkt
`scalar_bounds`	`smin>=0 && umax<=limit`	unbounded min value
`reference_release`	`all refs released before exit`	Unreleased reference id=N
`context_access`	`off+sz<=ctx_size`	invalid bpf_context access
`alignment`	`off % align == 0`	misaligned access
`type_safety`	`type matches expected`	R1 inv expected fp

The primary challenge is format variability. Register state can appear before or after the instruction, in branch merge lines, in backtracking annotations, or across multiple lines. The parser normalizes all variants into a uniform `TracedInstruction` representation.

Backtracking chain extraction. The verifier's own `mark_precise` backward pass is emitted in the log as `regs=R stack=S before K: (op) ...` annotations. OBLIGE extracts these into a `BacktrackChain` structure: a sequence of (instruction_index, registers_tracked) pairs. This is the verifier's own root-cause chain, expressed as debug text; OBLIGE is the first tool to extract and structure it.

3.3 Proof Obligation Inference

Given the error message and the register state at the rejection point, OBLIGE infers what the verifier *needed* to see: the *proof obligation*. This is expressed as a formal predicate over register state.

OBLIGE supports eighteen obligation families, shown in Table 1. The error message identifies the obligation *family*. The register state at the rejection point provides concrete parameters. OBLIGE instantiates the predicate template with these parameters.

This predicate instantiation is not pattern matching on error text. The same error message (`invalid access to packet`) with different register states at the rejection point produces different predicate instances with different parameters. The predicate is grounded in the verifier's type system: for packet access, the verifier literally checks `reg.off + access_size <= reg.range`; OBLIGE infers this from the register fields at the rejection point.

Inference algorithm. For each error ID, OBLIGE defines: (1) which register(s) contain the failing object (extracted from the rejection line), (2) which predicate template applies (from the obligation family), and (3) how to extract concrete parameters from the register state dump. The resulting predicate is a boolean function over register state that can be evaluated at any point in the trace.

3.4 Formal Predicate Tracking

Given the instantiated predicate P from Section 3.3, OBLIGE evaluates P at every instruction in the trace. This produces a boolean sequence over the trace:

$$\underbrace{?, \ldots, ?}_{\text{unknown}}, \underbrace{\top, \ldots, \top}_{\text{satisfied}}, \underbrace{\bot, \ldots, \bot}_{\text{violated}}$$

OBLIGE assigns proof lifecycle labels to each instruction:

- *proof_established*: the first instruction where P becomes satisfied (? → `true`)
- *proof_holds*: instructions where P remains satisfied
- *proof_lost*: the first instruction where P transitions from satisfied to violated (`true` → `false`)—the *transition witness*
- *rejected*: the final error instruction (always identified)

If P is never satisfied, the status is *never_established* (a source bug: the proof obligation was never met, indicating a missing safety check). If P is satisfied but becomes violated before the rejection, the status is *established_then_lost* (a lowering artifact or proof propagation failure: the source has a valid check but the lowered code loses it).

Key distinction from heuristic approaches. The transition witness is not "detect a bounds collapse pattern." It is "evaluate the specific predicate that the verifier needed at this specific rejection point, and find the exact instruction where it broke." Different predicates produce different transition witnesses on the same trace. This is formally grounded in the verifier's type system, not in ad hoc pattern detection.

Register value lineage. Predicates reference registers, but register names are reused across the trace. A value computed in R0 may be copied to R3 before the access. OBLIGE tracks value identity across register moves and copies. If R3 is defined as a copy of R0 (`r3 = r0`), and the obligation predicate references the packet range of R0, OBLIGE continues tracking the range on R3 for subsequent instructions. This register value lineage is essential for correctly identifying proof-established and proof-lost events across LLVM's register allocation.

Proof propagation analysis. For lowering artifact detection, OBLIGE additionally checks whether the proof established at the bounds-check site *propagates* to the register used at the access site. If a different register is used for the access than the one whose range was established at the check (a common LLVM pattern), OBLIGE classifies this as a lowering artifact regardless of whether the proof predicate was technically satisfied at some earlier instruction.

3.5 Formal Foundation: Meta-Analysis of Abstract Interpretation Output

The predicate tracking described above can be understood as a *second-order abstract interpretation*: OBLIGE applies abstract interpretation to the *output* of another abstract interpreter [1]. This section formalizes the framework.

Two layers of abstract interpretation. Layer 1 is the eBPF verifier itself. Its input is BPF bytecode; its abstract domain D_1 consists of register types, scalar bounds [`umin`, `umax`], pointer offsets, and packet ranges. Its output is a trace $T = (s_0, s_1, \ldots, s_n)$ where each $s_i \in D_1$ is the abstract state after the i-th instruction.

Layer 2 is OBLIGE. Its input is the trace T; its abstract domain D_2 tracks proof obligation status; its output is a sequence of proof lifecycle labels over T.

Definition 1 (Obligation status lattice). *The obligation status lattice is $\mathcal{L} = \{\bot, unknown, satisfied, violated\}$ with ordering $\bot \sqsubseteq unknown \sqsubseteq satisfied$ and $\bot \sqsubseteq unknown \sqsubseteq violated$, where satisfied and violated are incomparable. The bottom element $\bot$ represents an uninitialized status (obligation predicate not yet evaluable); unknown represents a state where the predicate is evaluable but the relevant registers are not yet constrained.*

Definition 2 (Obligation transfer function). *Let $P : D_1 \to \{true, false, unknown\}$ be the obligation predicate instantiated from the rejection point (§3.3). For each traced instruction i with verifier state $s_i \in D_1$, the obligation transfer function $\tau_i : \mathcal{L} \to \mathcal{L}$ is:*

$$\tau_i(\ell) = \begin{cases} sat & P(s_i) \wedge \ell \in \{unk, sat\} \\ vio & \neg P(s_i) \wedge \ell = sat \\ \ell & otherwise \end{cases}$$

The status sequence is computed by forward iteration: $\ell_0 = unknown,\ \ell_i = \tau_i(\ell_{i-1})$.

Definition 3 (Transition witness). *The transition witness is the smallest index w such that $\ell_{w-1} = satisfied$ and $\tau_w(\ell_{w-1}) = violated$. This is the exact instruction where the proof breaks. If no such w exists and $\ell_n = violated$, the proof was* never established.

Proposition 1 (Soundness of lifecycle labels). *If OBLIGE labels instruction i with $\ell_i = satisfied$, then for all concrete states $c \in \gamma(s_i)$ (where γ is the concretization function of the verifier's abstract domain), the safety property encoded by P holds.*

Proof sketch. The verifier is a sound abstract interpreter: s_i over-approximates all concrete states reachable at instruction i [4]. The obligation predicate P is monotone with respect to the verifier's partial order on D_1 (tighter bounds imply satisfaction if looser bounds already satisfy). Therefore $P(s_i) = true$ implies $P(c) = true$ for all $c \in \gamma(s_i)$. □

Backward obligation slice. Given the transition witness w, the *backward obligation slice* is

$$S = \{i < w \mid \exists\, r \in regs(P).\ \text{instruction } i \text{ writes to } r\}$$

i.e., all instructions before w that write to registers appearing in the obligation predicate P. When `mark_precise` backtracking information is available in the trace, S is refined to include only instructions on the backtracking chain, yielding a tighter slice.

Generalization beyond eBPF. This framework applies to any abstract interpreter that outputs per-step abstract states. The requirements are: (1) per-step abstract states are emitted, (2) safety properties are expressible as predicates over the abstract domain, and (3) the trace is ordered. Rust's borrow checker (lifetime predicates over borrow state), WebAssembly validators (type predicates over operand stack state), and Java bytecode verifiers (type predicates over stack frame state) all satisfy these requirements—suggesting that second-order trace analysis may be a general technique for improving diagnostic quality in verified language runtimes.

3.6 BTF Source Correlation

OBLIGE maps each proof lifecycle event (established, lost, rejected) to its source location via BTF `line_info` annotations [8]. When the verifier emits a source-location comment before an instruction (e.g., `; foo.c:42`), OBLIGE records the mapping. Multiple consecutive bytecode instructions from the same source line are merged into a single span.

The result is 3–5 source-level spans, each labeled with a proof lifecycle role and annotated with the register state change at that point. When BTF annotations are absent (37.3% of our corpus, primarily Stack Overflow cases where log fragments lack BTF context), OBLIGE falls back to bytecode-level spans (instruction indices only).

BTF coverage is a data quality issue, not a parser limitation: kernel selftest cases have 98.7% BTF coverage, confirming that the correlation works correctly when the input has BTF annotations.

3.7 Diagnostic Rendering

OBLIGE produces two output formats from the same proof lifecycle data:

Human-readable Rust-style output. Modeled on Rust compiler diagnostics [12], the output shows multiple source spans with causal labels, register state changes at each transition, and `note`/`help` annotations. The format is designed to be readable in a terminal, an IDE, or a CI log without any tool support.

Structured JSON. A machine-readable format containing `error_id`, `taxonomy_class`, `proof_status`, an array of `spans` (each with role, source location, instruction index, and state change), the instantiated obligation predicate, and `note`/`help` text. This format is designed for consumption by LLM agents (replacing raw log text in prompts), CI systems (structured failure annotations), and IDE plugins.

3.8 Implementation Notes

OBLIGE is implemented in Python (~6,600 lines across five modules: `log_parser`, `trace_parser`, `proof_engine`, `source_correlator`, and `rust_diagnostic`), with 120

Table 2. Evaluation corpus.

Source	Cases	Eligible	BTF	Known fix
Kernel selftests	200	150	98.7%	200
Stack Overflow	76	65	1.5%	66
GitHub issues	26	26	7.7%	15
Total	**302**	**241**	**62.7%**	**281**

unit tests covering obligation inference, predicate evaluation, backward slicing, and transition witness detection on hand-crafted and real-corpus traces. It defines 23 error IDs (OBLIGE-E001 through E023) and eighteen obligation families.

Source-code independence. OBLIGE requires only the verifier log text as input—no object files, source files, or external debug information. When BTF `line_info` annotations are present in the log, OBLIGE maps proof events to source-level spans; when BTF is absent, it falls back to bytecode-indexed spans. For already-loaded programs, `bpftool prog dump xlated linum` can supply supplementary source context, but this is optional.

Language independence. Because OBLIGE analyzes the verifier trace at the BPF bytecode level, it is agnostic to the source language. We validated the pipeline on 18 Rust (Aya) and 7 Go (Cilium) cases alongside the C corpus: diagnostic generation succeeds on all 25 cases, with obligation inference on 17/25 (68%) and taxonomy classification on 25/25 (100%). The lower obligation rate for non-C cases reflects thinner logs in the stored corpus, not a language-specific limitation.

4 Evaluation

We evaluate OBLIGE against six questions:

- **Q1** Does OBLIGE generate diagnostics reliably at scale?
- **Q2** What fraction of cases get full proof obligation coverage?
- **Q3** Do OBLIGE's source spans cover actual fix locations?
- **Q4** Do OBLIGE diagnostics improve LLM-assisted repair?
- **Q5** How does OBLIGE compare to Pretty Verifier?
- **Q6** What is the runtime overhead?

4.1 Corpus and Methodology

Table 2 describes our evaluation corpus. The primary corpus comprises 302 real verifier failure cases from three sources: 200 kernel selftests (with `__msg()` annotations specifying expected verifier errors), 76 Stack Overflow questions with attached verifier logs, and 26 GitHub issues from Cilium, Aya, and Katran. Of the 302 cases, 241 have verifier logs of sufficient length for diagnostic evaluation (≥50 characters); these form our evaluation corpus.

Thirty cases are manually labeled with ground-truth taxonomy by a domain expert (the first author), achieving Cohen's $\kappa = 0.652$ against the heuristic classifier. The heuristic classifier agrees with manual labels in 76.7% of cases overall, but only 33.3% on lowering artifacts—confirming that this is the hardest class to diagnose from error messages alone.

Table 3. Batch diagnostic evaluation on 241 cases (Q1, Q2).

Metric	Result
Diagnostic generation success	241/241 (100%)
Proof obligation inferred	227/241 (94.2%)
BTF source correlation	151/241 (62.7%)
Proof-established span	103/241 (42.7%)
Proof-lost span	90/241 (37.3%)
Proof status: never_established	117/241 (48.5%)
Proof status: established_then_lost	90/241 (37.3%)

The taxonomy distribution across the 241-case evaluation corpus: source_bug 45.2%, env_mismatch 34.4%, lowering_artifact 12.0%, verifier_limit 8.3%.

4.2 Batch Reliability and Obligation Coverage (Q1, Q2)

We ran OBLIGE on all 241 eligible cases. Table 3 reports the results.

OBLIGE generates a structured diagnostic for every case with zero crashes. Proof obligation coverage of 94.2% (227/241) means that for nineteen in twenty cases, OBLIGE can state the exact predicate the verifier needed and evaluate it at every instruction. The remaining 6% involve environment errors (Permission denied, build failures) rather than true verifier rejections, and fall outside the eighteen currently supported obligation families. We explicitly characterize this scope limitation in Section 5.

The proof status distribution reflects the failure mix: 48.5% are *never_established* (source bugs with no proof at any point), 37.3% are *established_then_lost* (proofs that exist in source but are destroyed during lowering or across program structure). This 37.3% is the population where OBLIGE provides its strongest differentiating value.

Span coverage (Q3). For cases with known fixes, 12/14 manually annotated cases (85.7%) have at least one OBLIGE span overlapping the fix location; on kernel selftests, 85/102 (83.3%) of rejected spans match the expected error instruction. The overall 38.4% (101/263) figure is deflated by Stack Overflow cases where fix locations are described in prose rather than as line numbers—a corpus limitation, not a parser one.

4.3 LLM Repair Experiment (Q4)

We evaluate whether OBLIGE diagnostics help an LLM generate better repairs. We use an A/B design: the same LLM (GPT-4.1-mini) receives either (A) buggy code plus raw verifier log, or (B) buggy code plus raw log plus OBLIGE's Rust-style diagnostic. We measure fix-type accuracy (does the LLM choose the correct type of fix?), root-cause targeting

Table 4. A/B LLM repair experiment (54 cases). Condition A: raw verifier log only. Condition B: raw log + OBLIGE diagnostic.

Metric	A (raw log)	B (+OBLIGE)	Δ
Overall location accuracy	53/54 (98.1%)	48/54 (88.9%)	−9pp
Overall fix-type accuracy	46/54 (85.2%)	43/54 (79.6%)	−6pp
Lowering fix type	**3/10 (30%)**	**6/10 (60%)**	**+30pp**
Source bug fix type	27/28 (96.4%)	23/28 (82.1%)	−14pp
Verifier limit fix type	8/8 (100%)	7/8 (87.5%)	−12.5pp
Env mismatch fix type	8/8 (100%)	7/8 (87.5%)	−12.5pp

(does it target the right location?), and semantic similarity to the known correct fix.

Table 4 shows results on 54 cases (10 lowering artifacts, 28 source bugs, 8 verifier limits, 8 env mismatches).

The headline overall fix-type accuracy shows Condition A marginally ahead (85.2% vs 79.6%), but the per-class breakdown tells the story. On lowering artifacts—the class where raw error messages are most misleading—OBLIGE diagnostics double the fix-type accuracy: 3/10 (30%) with raw logs versus 6/10 (60%) with OBLIGE diagnostics, a +30 pp gain. In Condition A, the LLM sees an error like "unbounded min value" and adds another bounds check (wrong). In Condition B, it sees that the bounds check at line 3 *established* the proof, the `htons` call at line 7 *destroyed* it, and it correctly applies a scalar clamp.

The overall regression on non-lowering classes (Condition B slightly below A) has a known root cause: several source-bug cases received OBLIGE output pointing to a BTF metadata issue rather than the actual logic fix (e.g., BTF/-func_info regeneration advice on dynptr and iterator cases), pushing the LLM away from the correct repair. The raw log for these cases already named the specific contract violation, and the generic OBLIGE output added noise. This is a known issue with BTF-absent traces and is targeted for improvement.

We present this as evidence that OBLIGE's value is concentrated on the lowering-artifact class, where proof-trace analysis provides qualitatively different information not available in raw logs.

4.4 Comparison with Pretty Verifier (Q5)

We compared OBLIGE against Pretty Verifier on the 30 manually labeled cases. Table 5 summarizes the results.

The key distinction is not classification accuracy. It is root-cause localization. Pretty Verifier finds root cause in 0/30 cases because it does not analyze state evolution. OBLIGE finds an earlier proof-loss site in 12/30 cases (40%)—and in 4/4 of these cases, the proof-loss site is a *different instruction* than the final rejection, providing qualitatively different guidance.

On lowering artifacts specifically, Pretty Verifier crashes with a Python exception on 3/6 cases (due to brittle string indexing on multi-line log endings) and correctly diagnoses only 1/6. OBLIGE correctly diagnoses 4/6. The two remaining OBLIGE failures are due to absent state traces (one case has only a 7-instruction snippet, and one requires cross-subprogram analysis not yet implemented).

Table 5. OBLIGE vs. Pretty Verifier on 30 manually labeled cases.

Metric	Pretty Verifier	OBLIGE
Overall classification accuracy	19/30 (63%)	25/30 (83%)
Lowering artifact accuracy	1/6 (17%)	4/6 (67%)
Root-cause localization	0/30 (0%)	12/30 (40%)
Cases with actionable diagnosis	5/30 (17%)	20/30 (67%)
Crash-free operation	22/30 (73%)	30/30 (100%)

Source localization also differs fundamentally. Pretty Verifier maps source via `llvm-objdump` on the compiled object file (unavailable in most deployment contexts). OBLIGE uses BTF annotations embedded in the log itself, achieving 62.7% coverage without any external files.

4.5 Runtime Overhead (Q6)

OBLIGE processes all 241 cases with median latency 27 ms (P95: 43 ms, max: 92 ms, scaling linearly with log length at Pearson $r = 0.74$). All cases complete in under 100 ms—comfortably within interactive bounds and negligible compared to a single LLM API call (1–10 s).

5 Discussion and Limitations

5.1 Scope and Honest Limitations

94% obligation coverage. Eighteen obligation families cover 94.2% of cases. The remaining 6% involve environment errors (Permission denied, build failures) rather than true verifier rejections; these fall outside the scope of proof-trace analysis. The predicate framework is extensible, and adding a new family requires defining the predicate template and the parameter extraction rules. We provide an explicit enumeration of the eighteen supported families (Table 1) so readers can assess scope.

BTF coverage. 37.3% of cases lack BTF annotations (primarily Stack Overflow fragments). Without BTF, OBLIGE produces bytecode-level spans (instruction indices) rather than source-level spans. This degrades human readability but not the underlying predicate analysis. The 98.7% BTF coverage on kernel selftests confirms that the correlation is correct when inputs are well-formed.

Path-insensitive slicing. OBLIGE's backward slice from the transition witness is bounded and path-insensitive. It will not correctly handle all loop- or alias-sensitive proof loss patterns. We scope the slice as an explanatory aid after the main predicate/transition detection, and validate it empirically rather than claiming formal precision.

LLM repair experiment uses proxy metrics. The A/B experiment uses 54 cases and proxy metrics (fix-type classification) rather than verifier-pass outcomes. The regression on source-bug cases caused by BTF-absent traces pointing toward metadata issues is a known failure mode. A full re-run with updated prompts and a verifier-pass oracle is planned.

No cross-kernel stability evaluation. The verifier's log format varies across kernel versions. We have not yet evaluated OBLIGE across multiple kernel versions. Prior work [15] finds that 83% of eBPF extensions are affected by at least one kernel mismatch; this is an important axis for future work.

5.2 Upstream Potential

OBLIGE's output could become part of the kernel's own diagnostic infrastructure. The structured JSON format and 23 error IDs are designed to be stable and extensible. The proof obligation catalog could inform improvements to verifier error messages. The Rust-style rendering could be integrated into `bpftool` or the `libbpf` loader, so developers receive structured diagnostics directly from the load path.

6 Related Work

6.1 eBPF Verifier Complexity and Developer Pain

Jia et al. [7] argue that kernel-extension verification is "untenable" as the extension surface grows; OBLIGE addresses the near-term reality that millions of C/Rust eBPF programs run against the current verifier. Rex [6] studies 72 verifier workaround commits, confirming 44.4% are LLVM code-generation changes; Rex explains *why* such failures exist, OBLIGE explains *this specific failure instance* at load time. Deokar et al. [2] find 19.3% of 743 Stack Overflow questions are verifier-related, quantifying the pain point OBLIGE addresses.

6.2 Diagnostic Tools

Pretty Verifier [9] applies 91 regex patterns to the final verifier error line, producing a single-span explanation; it cannot detect proof-loss transitions and crashes on 10.7% of our corpus. OBLIGE processes the complete state trace with formal predicate evaluation—a different level of analysis that enables root-cause localization and lowering artifact diagnosis. The `ebpf-verifier-errors` repository [11] crowdsources (log, fix) pairs as data, but provides no automated analysis.

6.3 LLM-Based eBPF Tools

Kgent [16] feeds raw verifier output verbatim to an LLM repair loop; OBLIGE's structured diagnostics could serve as Kgent's preprocessor, replacing unstructured register dumps with labeled causal spans. SimpleBPF [3] generates a high-level DSL and explicitly notes that raw-log retry "was not effective"; OBLIGE addresses the general case (arbitrary hand-written C/Rust) that DSLs cannot replace. K2 [14] optimizes eBPF programs for performance, a goal orthogonal to failure diagnosis.

6.4 Static Analysis and Diagnostics

OBLIGE's approach is inspired by counterexample analysis in model checking: reconstruct why verification failed from the evidence trace. Abstract interpretation [1] and its formalization for eBPF [4] ground OBLIGE's obligation predicates. Program slicing [13] motivates backward dependency extraction from the transition witness. Rust's compiler diagnostics [12] pioneered multi-span source-level errors with causal labels; research on compiler error messages [5] confirms that pointing at the wrong location significantly harms developer productivity.

7 Conclusion

The eBPF verifier already emits—in its `LOG_LEVEL2` trace—everything needed to understand why a program is rejected. OBLIGE demonstrates that meta-analysis of this abstract interpretation output enables Rust-quality multi-span diagnostics: formal predicate tracking to identify exactly where proof obligations are established, propagated, and lost. The clearest payoff is for lowering artifacts, where line-oriented tools localize root causes in 0% of cases; OBLIGE correctly diagnoses 67% and doubles LLM repair accuracy, enabling correct repairs that the raw log cannot guide. As eBPF programs grow in complexity, structured proof trace analysis at the scale and latency demonstrated here (27 ms, no kernel patches) is a practical path to making verifier failures comprehensible.

References

[1] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. *SIGPLAN Notices* (1977), 238–252. https://doi.org/10.1145/512950.512973

[2] Mugdha Deokar, Jingyang Men, Lucas Castanheira, Ayush Bhardwaj, and Theophilus A. Benson. 2024. An Empirical Study on the Challenges of eBPF Application Development. In *Proceedings of the SIGCOMM Workshop on eBPF and Kernel Extensions*. 1–8. https://doi.org/10.1145/3672197.3673429

[3] Xiangyu Gao, Xiangfeng Zhu, Bhavana Vannarth Shobhana, Yiwei Yang, Arvind Krishnamurthy, and Ratul Mahajan. 2025. Offloading the Tedious Task of Writing eBPF Programs. In *Proceedings of the 3rd Workshop on eBPF and Kernel Extensions*. 63–69. https://doi.org/10.1145/3748355.3748369

[4] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzky, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 1–15. https://doi.org/10.1145/3314221.3314590

[5] Brett Hartmann, Mark Deline, and Peter Lee. 2014. Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research. In *Proceedings of the Workshop in Primary and Secondary Computing Education (WiPSCE)*. 75–84. https://doi.org/10.1145/2670757.2670780

[6] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukiyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: Closing the Language-Verifier Gap with Safe and Usable Kernel Extensions. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, 325–342. https://www.usenix.org/conference/atc25/presentation/jia

[7] Jinghao Jia, Raj Sahu, Adam Oswald, Dan Williams, Michael V. Le, and Tianyin Xu. 2023. Kernel Extension Verification is Untenable. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HotOS '23)*. 150–157. https://doi.org/10.1145/3593856.3595892

[8] Linux Kernel Documentation. 2024. BPF Type Format (BTF). Linux kernel documentation. https://docs.kernel.org/bpf/btf.html

[9] Sebastiano Miano, Roberto Lezzi, Gabriele Lospinoso, and Fulvio Risso. 2025. Pretty Verifier: Towards Friendly eBPF Verification Errors. GitHub repository. https://github.com/netgroup-polito/pretty-verifier Unpublished, accessed 2026-03-12.

[10] NCC Group. 2024. *eBPF Verifier Code Audit*. Technical Report. The Linux Foundation. https://www.linuxfoundation.org/press/linux-foundation-releases-open-source-technology-security-audit-of-the-ebpf-verifier

[11] parttimenerd. 2024. ebpf-verifier-errors. GitHub repository. https://github.com/parttimenerd/ebpf-verifier-errors Accessed 2026-03-12.

[12] Rust Language Team. 2016. Improved Error Messages in the Rust Compiler. The Rust Blog. https://blog.rust-lang.org/2016/08/10/Shape-of-errors-to-come.html Accessed 2026-03-12.

[13] Mark Weiser. 1981. Program Slicing. In *Proceedings of the 5th International Conference on Software Engineering (ICSE)*. 439–449.

[14] Qiongwen Xu, Michael D. Brown, Tao Li, Chengbin Pang, Bhavna Sopori, Sahil Jain, Junfeng Yang, and Arpit Gupta. 2023. K2: Automated eBPF Program Optimization. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 1–15. https://doi.org/10.1145/3603269.3604882

[15] Junxiao Zhang, Samantha Mathews, Hiep Nguyen, Ryan Stutsman, and Asaf Cidon. 2025. DepSurf: Measuring and Analyzing the Dependency Surface of Kernel Extensions. Project site and paper. https://sites.google.com/view/depsurf-bpf/home

[16] Yusheng Zheng, Yiwei Yang, Maolin Chen, and Andrew Quinn. 2024. Kgent: Kernel Extensions Large Language Model Agent. In *Proceedings of the SIGCOMM Workshop on eBPF and Kernel Extensions*. 30–36. https://doi.org/10.1145/3672197.3673434